Exercises: Gradient Methods and Optimization Convergence – Answer Key

Mathematical Foundations for Machine Learning

Gradient Descent Fundamentals

Exercise 1.1 [Easy]
Let $f : \mathbb{R}^d \to \mathbb{R}$ be continuously differentiable with L-Lipschitz continuous gradient, i.e.,

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2 \quad \forall\, x, y \in \mathbb{R}^d.$$

(a) Show that the Lipschitz condition on the gradient implies the following *descent lemma*:

$$f(y) \leq f(x) + \langle \nabla f(x), y - x\rangle + \frac{L}{2}\|y - x\|_2^2 \quad \forall\, x, y \in \mathbb{R}^d.$$

(Hint: write $f(y) - f(x) = \int_0^1 \langle \nabla f(x + t(y - x)),\, y - x\rangle\, dt$ and use Cauchy–Schwarz.)

(b) Using part (a), substitute $y = x - \frac{1}{L}\nabla f(x)$ (one gradient step with step size $t = \frac{1}{L}$). Show that

$$f\big(x - \tfrac{1}{L}\nabla f(x)\big) \leq f(x) - \frac{1}{2L}\|\nabla f(x)\|_2^2.$$

(c) Explain why part (b) guarantees that gradient descent with step size $t = \frac{1}{L}$ produces a *monotonically decreasing* sequence of objective values.

Solution

(a) By the fundamental theorem of calculus:

$$f(y) - f(x) = \int_0^1 \langle \nabla f(x + t(y - x)),\, y - x\rangle\, dt.$$

Adding and subtracting $\langle \nabla f(x), y - x\rangle$:

$$f(y) - f(x) = \langle \nabla f(x), y - x\rangle + \int_0^1 \langle \nabla f(x + t(y - x)) - \nabla f(x),\, y - x\rangle\, dt.$$

By Cauchy–Schwarz and the Lipschitz condition on ∇f:

$$|\langle \nabla f(x+t(y-x)) - \nabla f(x),\, y-x\rangle| \leq \|\nabla f(x+t(y-x)) - \nabla f(x)\|_2 \cdot \|y-x\|_2 \leq Lt\|y-x\|_2^2.$$

Integrating: $\int_0^1 Lt\, dt = \frac{L}{2}$, which gives the result.

(b) Set $y = x - \frac{1}{L}\nabla f(x)$ in part (a). Then $y - x = -\frac{1}{L}\nabla f(x)$, so:

$$f(y) \leq f(x) + \big\langle \nabla f(x),\, -\tfrac{1}{L}\nabla f(x)\big\rangle + \frac{L}{2}\left\|\tfrac{1}{L}\nabla f(x)\right\|_2^2 = f(x) - \frac{1}{L}\|\nabla f(x)\|_2^2 + \frac{1}{2L}\|\nabla f(x)\|_2^2 = f(x) - \frac{1}{2L}\|\nabla f(x)\|_2^2.$$

(c) Since $\|\nabla f(x)\|_2^2 \geq 0$, part (b) gives $f(x^{(k+1)}) \leq f(x^{(k)})$ at every step. The decrease is strict whenever $\nabla f(x^{(k)}) \neq 0$, so the sequence $\{f(x^{(k)})\}$ is monotonically non-increasing and strictly decreasing until a critical point is reached.

Exercise 1.2 [Easy]
Consider the quadratic function $f(x) = \frac{1}{2}x^T Ax - b^T x$ where $A \in \mathbb{R}^{d\times d}$ is symmetric positive definite with eigenvalues $0 < \lambda_1 \leq \cdots \leq \lambda_d$.

(a) Compute the gradient $\nabla f(x)$ and the Hessian $\nabla^2 f(x)$.

(b) Show that ∇f is Lipschitz continuous with Lipschitz constant $L = \lambda_d$ (the largest eigenvalue).

(c) Show that f is λ_1-strongly convex.

(d) Write down the gradient descent update $x^{(k+1)} = x^{(k)} - t\,\nabla f(x^{(k)})$ and simplify it. For what range of step sizes t does the iteration converge?

Solution

(a) $\nabla f(x) = Ax - b$ and $\nabla^2 f(x) = A$.
(b) $\|\nabla f(x) - \nabla f(y)\| = \|A(x-y)\| \leq \|A\|_2\|x-y\| = \lambda_d\|x-y\|$, where we used that the operator norm of a symmetric matrix equals its largest eigenvalue.
(c) $\nabla^2 f(x) = A \succeq \lambda_1 I \succ 0$ for all x, which is the second-order characterization of λ_1-strong convexity.
(d) $x^{(k+1)} = x^{(k)} - t(Ax^{(k)} - b) = (I - tA)x^{(k)} + tb$. Convergence requires $\|I - tA\|_2 < 1$, i.e., $|1 - t\lambda_i| < 1$ for all i. This gives $0 < t < \frac{2}{\lambda_d}$. The optimal step size minimizing $\max_i |1 - t\lambda_i|$ is $t^* = \frac{2}{\lambda_1 + \lambda_d}$.

Exercise 1.3 [Medium]
Let $f : \mathbb{R}^d \to \mathbb{R}$ be convex with L-Lipschitz continuous gradient, and let x^* be a minimizer.

(a) Starting from the descent lemma

$$f(x^{(k+1)}) \leq f(x^*) + \langle \nabla f(x^{(k)}),\, x^{(k)} - x^* \rangle - \frac{t}{2}\|\nabla f(x^{(k)})\|_2^2,$$

show that the right-hand side can be rewritten as

$$f(x^*) + \frac{1}{2t}\left(\|x^{(k)} - x^*\|_2^2 - \|x^{(k+1)} - x^*\|_2^2\right).$$

(Hint: complete the square.)

(b) Using part (a) and the fact that $f(x^{(k)})$ is non-increasing, prove that

$$f(x^{(k)}) - f(x^*) \leq \frac{\|x^{(0)} - x^*\|_2^2}{2kt}.$$

(Hint: average over iterations $1, \ldots, k$ and use telescoping.)

(c) A neural network's loss landscape has $L = 50$. Using part (b) with step size $t = 1/L$, how many gradient steps are needed to guarantee $f(x^{(k)}) - f(x^*) \leq 0.01$, given $\|x^{(0)} - x^*\|_2 = 10$?

Solution

(a) Using $x^{(k+1)} = x^{(k)} - t\nabla f(x^{(k)})$, we have $\nabla f(x^{(k)}) = \frac{1}{t}(x^{(k)} - x^{(k+1)})$. The key identity is:

$$\langle \nabla f(x^{(k)}), x^{(k)} - x^* \rangle - \frac{t}{2}\|\nabla f(x^{(k)})\|^2 = \frac{1}{2t}\left(\|x^{(k)} - x^*\|^2 - \|x^{(k)} - x^* - t\nabla f(x^{(k)})\|^2\right)$$

$$= \frac{1}{2t}\left(\|x^{(k)} - x^*\|^2 - \|x^{(k+1)} - x^*\|^2\right).$$

This follows from expanding $\|x^{(k+1)} - x^*\|^2 = \|x^{(k)} - x^* - t\nabla f(x^{(k)})\|^2$.
(b) From part (a): $f(x^{(k+1)}) - f(x^*) \leq \frac{1}{2t}(\|x^{(k)} - x^*\|^2 - \|x^{(k+1)} - x^*\|^2)$. Summing from $k = 0$ to $k - 1$:

$$\sum_{s=0}^{k-1}(f(x^{(s+1)}) - f(x^*)) \leq \frac{1}{2t}(\|x^{(0)} - x^*\|^2 - \|x^{(k)} - x^*\|^2) \leq \frac{\|x^{(0)} - x^*\|^2}{2t}.$$

Since $f(x^{(k)}) \leq f(x^{(s)})$ for $s \leq k$, we get $k(f(x^{(k)}) - f(x^*)) \leq \frac{\|x^{(0)} - x^*\|^2}{2t}$.
(c) With $t = 1/L = 1/50$: need $\frac{L\|x^{(0)} - x^*\|^2}{2k} \leq 0.01$, so $k \geq \frac{50 \cdot 100}{0.02} = 250{,}000$ steps.

Exercise 1.4 [Medium]
In gradient descent with exact line search, the step size is chosen as

$$\alpha_k = \arg\min_{\alpha \geq 0} f(x^{(k)} - \alpha\, \nabla f(x^{(k)})).$$

(a) Let $\phi(\alpha) = f(x^{(k)} - \alpha\, \nabla f(x^{(k)}))$. Using the chain rule, show that

$$\phi'(\alpha) = -\langle \nabla f(x^{(k)} - \alpha\, \nabla f(x^{(k)})),\, \nabla f(x^{(k)}) \rangle.$$

(b) Prove that consecutive gradient descent directions with exact line search are orthogonal:

$$\langle \nabla f(x^{(k+1)}),\, \nabla f(x^{(k)}) \rangle = 0.$$

(c) Explain why this orthogonality property means steepest descent with exact line search produces a characteristic *zig-zag* pattern. Is this desirable for convergence?

Solution

(a) Define $g(\alpha) = x^{(k)} - \alpha\, \nabla f(x^{(k)})$. Then $\phi(\alpha) = f(g(\alpha))$ and by the chain rule:

$$\phi'(\alpha) = \langle \nabla f(g(\alpha)),\, g'(\alpha) \rangle = \langle \nabla f(x^{(k)} - \alpha\, \nabla f(x^{(k)})),\, -\nabla f(x^{(k)}) \rangle.$$

(b) At the optimal α_k, we have $\phi'(\alpha_k) = 0$. Since $x^{(k+1)} = x^{(k)} - \alpha_k \nabla f(x^{(k)})$:

$$0 = \phi'(\alpha_k) = -\langle \nabla f(x^{(k+1)}),\, \nabla f(x^{(k)}) \rangle.$$

(c) Each step moves in the steepest descent direction, which is orthogonal to the previous step direction. This creates a zig-zag pattern where the iterates bounce back and forth between alternating directions. For ill-conditioned problems (large $\lambda_{\max}/\lambda_{\min}$), the narrow valleys cause the iterates to oscillate across the valley rather than moving along it, leading to very slow convergence. This is undesirable and motivates methods like conjugate gradients, momentum, or quasi-Newton methods that correct for this behavior.

Lipschitz Gradients and the Polyak-Łojasiewicz Condition

Exercise 2.1 [Easy]
Let $f(x) = \frac{1}{2}\|Ax - b\|_2^2$ for $A \in \mathbb{R}^{m\times d}$ and $b \in \mathbb{R}^m$.

(a) Compute $\nabla f(x)$ and determine the Lipschitz constant L of the gradient.

(b) Is f strongly convex? If so, with what constant? Under what condition on A?

(c) What is the condition number $\kappa = L/m$ of this problem? Interpret what happens to gradient descent convergence when $\kappa \gg 1$.

Solution

(a) $\nabla f(x) = A^T(Ax - b)$ and $\nabla^2 f(x) = A^T A$. The Lipschitz constant of the gradient is $L = \|A^T A\|_2 = \sigma_{\max}(A)^2$ (the largest squared singular value of A).
(b) f is m-strongly convex with $m = \lambda_{\min}(A^T A) = \sigma_{\min}(A)^2$ if and only if A has full column rank (i.e., $\sigma_{\min}(A) > 0$, which requires $m \geq d$).
(c) $\kappa = \frac{\sigma_{\max}(A)^2}{\sigma_{\min}(A)^2} = \left(\frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}\right)^2 = \kappa(A)^2$, the square of the condition number of A. When $\kappa \gg 1$, the contours of f are very elongated ellipsoids. Gradient descent converges linearly with rate $(1 - 1/\kappa)$, so a large condition number means each step reduces the error by a tiny fraction, leading to very slow convergence.

Exercise 2.2 [Medium]
The **Polyak–Łojasiewicz (PL) condition** states that for some $\mu > 0$:

$$\frac{1}{2}\|\nabla f(x)\|_2^2 \geq \mu\,(f(x) - f(x^*)) \quad \forall\, x \in \mathbb{R}^d,$$

where $f(x^*) = \inf_x f(x)$.

(a) Show that every m-strongly convex function satisfies the PL condition with $\mu = m$. (Hint: minimize $f(y) \geq f(x) + \langle \nabla f(x), y - x\rangle + \frac{m}{2}\|y - x\|^2$ over y.)

(b) Consider $f(x) = (x_1^2 + x_2 - 1)^2$ on $\mathbb{R}^2$. The set of global minimizers is the parabola $\{(x_1, x_2) : x_2 = 1 - x_1^2\}$, which is non-convex and not a single point. Does f have a unique minimizer? Can a strongly convex function have multiple minimizers? Use this to explain why the PL condition is *strictly weaker* than strong convexity.

(c) Suppose f has L-Lipschitz continuous gradient and satisfies the PL condition with constant μ. Show that gradient descent with step size $t = 1/L$ converges linearly:

$$f(x^{(k+1)}) - f(x^*) \leq \left(1 - \frac{\mu}{L}\right)(f(x^{(k)}) - f(x^*)).$$

(Hint: use the sufficient decrease from the descent lemma and then apply PL.)

Solution

(a) For m-strongly convex f: $f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle + \frac{m}{2}\|y - x\|^2$ for all x, y. Minimizing the RHS over y: the optimal y satisfies $\nabla f(x) + m(y - x) = 0$, giving $y^* = x - \frac{1}{m}\nabla f(x)$. Substituting:

$$f(x^*) \leq f(y^*) \leq f(x) + \langle \nabla f(x), -\tfrac{1}{m}\nabla f(x) \rangle + \frac{m}{2} \cdot \frac{1}{m^2}\|\nabla f(x)\|^2 = f(x) - \frac{1}{2m}\|\nabla f(x)\|^2.$$

Rearranging: $\frac{1}{2}\|\nabla f(x)\|^2 \geq m(f(x) - f(x^*))$, which is PL with $\mu = m$.
(b) f has a continuum of minimizers (the parabola), so the minimizer is not unique. A strongly convex function has a *unique* minimizer (by strict convexity). Therefore f cannot be strongly convex, yet it may still satisfy PL. This shows PL is strictly weaker: it guarantees linear convergence of GD without requiring convexity or uniqueness of minimizers.
(c) From the descent lemma with $t = 1/L$:

$$f(x^{(k+1)}) \leq f(x^{(k)}) - \frac{1}{2L}\|\nabla f(x^{(k)})\|^2.$$

Applying PL: $\|\nabla f(x^{(k)})\|^2 \geq 2\mu(f(x^{(k)}) - f(x^*))$. Substituting:

$$f(x^{(k+1)}) \leq f(x^{(k)}) - \frac{\mu}{L}(f(x^{(k)}) - f(x^*)).$$

Subtracting $f(x^*)$:

$$f(x^{(k+1)}) - f(x^*) \leq (1 - \mu/L)(f(x^{(k)}) - f(x^*)).$$

Exercise 2.3 [Hard]
Consider training a linear model $h_\theta(x) = \theta^T x$ with the squared loss on data $\{(x_i, y_i)\}_{i=1}^N$:

$$f(\theta) = \frac{1}{2N}\sum_{i=1}^{N}(\theta^T x_i - y_i)^2 = \frac{1}{2N}\|X\theta - Y\|_2^2,$$

where $X \in \mathbb{R}^{N \times d}$. Assume $N > d$ and X has full column rank.

(a) Show that f satisfies the PL condition and find μ in terms of the singular values of X.

(b) Now consider the *overparameterized* regime $d > N$ (more parameters than data points). Does X have full column rank? Does f still satisfy the PL condition? Justify your answer.

(c) In the overparameterized case, f is not strongly convex yet gradient descent still converges linearly to a global minimum (of value 0). Explain how the PL condition accounts for this phenomenon, which is observed in training overparameterized neural networks.

Solution

(a) Since $N > d$ and X has full column rank, $A = \frac{1}{N}X^TX \succ 0$ with $\lambda_{\min}(A) = \sigma_{\min}(X)^2/N > 0$. The function $f(\theta) = \frac{1}{2}\theta^TA\theta - \frac{1}{N}X^TY \cdot \theta + \text{const}$ is $\lambda_{\min}(A)$-strongly convex, hence satisfies PL with $\mu = \sigma_{\min}(X)^2/N$.
(b) When $d > N$, $\text{rank}(X) \leq N < d$, so X does not have full column rank. Then $A = \frac{1}{N}X^TX$ has $\lambda_{\min} = 0$, and f is *not* strongly convex. However, if X has full row rank (rank N), then f still satisfies the PL condition with $\mu = \sigma_{\min,+}(X)^2/N > 0$, where $\sigma_{\min,+}$ is the smallest *nonzero* singular value. This is because $\nabla f(\theta) = \frac{1}{N}X^T(X\theta - Y)$, and $\|\nabla f\|^2 = \frac{1}{N^2}\|X^T(X\theta - Y)\|^2 \geq \frac{\sigma^2_{\min,+}}{N^2}\|X\theta - Y\|^2 = \frac{2\sigma^2_{\min,+}}{N}f(\theta)$.
(c) In the overparameterized regime, there are infinitely many global minimizers ($f = 0$ on an affine subspace). Strong convexity fails because of this non-uniqueness. However, PL still holds, guaranteeing that gradient descent converges linearly to *some* global minimizer. In neural network training, overparameterization creates a loss landscape where the gradient stays large relative to the suboptimality gap (the PL condition), allowing fast convergence even without convexity. This helps explain why SGD on overparameterized networks converges well in practice.

Newton's Method

Exercise 3.1 [Easy]
Newton's method for minimizing $f : \mathbb{R}^d \to \mathbb{R}$ (twice continuously differentiable) uses the update:

$$x^{(k+1)} = x^{(k)} - [\nabla^2 f(x^{(k)})]^{-1}\nabla f(x^{(k)}).$$

(a) Derive the Newton step by writing the second-order Taylor expansion of f around $x^{(k)}$, then minimizing this quadratic approximation with respect to the step Δx.

(b) What condition on $\nabla^2 f(x^{(k)})$ is needed for the Newton step to be well-defined and to be a descent direction?

(c) For $f(x) = x^4$, compute the Newton step at $x^{(0)} = 1$ and at $x^{(0)} = 0$. What goes wrong at $x^{(0)} = 0$?

Solution

(a) The second-order Taylor expansion:

$$f(x^{(k)} + \Delta x) \approx f(x^{(k)}) + \nabla f(x^{(k)})^T \Delta x + \frac{1}{2} \Delta x^T \nabla^2 f(x^{(k)}) \Delta x.$$

Minimizing over Δx: take the gradient w.r.t. Δx and set to zero:

$$\nabla f(x^{(k)}) + \nabla^2 f(x^{(k)}) \Delta x = 0 \implies \Delta x = -[\nabla^2 f(x^{(k)})]^{-1} \nabla f(x^{(k)}).$$

(b) $\nabla^2 f(x^{(k)})$ must be invertible (for the step to exist) and positive definite (for the quadratic model to be a minimizer, not a maximizer or saddle, ensuring Δx is a descent direction).
(c) $f'(x) = 4x^3$, $f''(x) = 12x^2$.
At $x^{(0)} = 1$: $\Delta x = -\frac{4}{12} = -\frac{1}{3}$, so $x^{(1)} = \frac{2}{3}$.
At $x^{(0)} = 0$: $f'(0) = 0$ and $f''(0) = 0$. The Hessian is singular, so the Newton step is undefined. The minimum is at $x = 0$, which is a degenerate critical point (the Hessian vanishes there).

Exercise 3.2 [Medium]
Consider the nonlinear least-squares problem

$$\min_{x \in \mathbb{R}^d} f(x) = \frac{1}{2} \|r(x)\|_2^2,$$

where $r : \mathbb{R}^d \to \mathbb{R}^m$ is the residual vector with Jacobian $J(x) = \frac{\partial r}{\partial x} \in \mathbb{R}^{m \times d}$.

(a) Show that $\nabla f(x) = J(x)^T r(x)$.

(b) Show that the Hessian is $\nabla^2 f(x) = J(x)^T J(x) + \sum_{i=1}^m r_i(x) \nabla^2 r_i(x)$.

(c) The **Gauss–Newton** method approximates $\nabla^2 f(x) \approx J(x)^T J(x)$ by dropping the second-order terms. Write down the Gauss–Newton update step. Under what condition on the residuals is this a good approximation?

(d) This is exactly the approach used in the Exam 2021, Exercise 2(c). Show that the Gauss–Newton step Δx solves the linear system $J^T J \Delta x = -J^T r$.

Solution

(a) Using the chain rule: $f(x) = \frac{1}{2} r(x)^T r(x)$, so

$$\frac{\partial f}{\partial x_j} = \sum_{i=1}^{m} r_i(x) \frac{\partial r_i}{\partial x_j} = [J(x)^T r(x)]_j.$$

Hence $\nabla f(x) = J(x)^T r(x)$.
(b) Differentiating again:

$$\frac{\partial^2 f}{\partial x_j \partial x_k} = \sum_{i=1}^{m} \left(\frac{\partial r_i}{\partial x_j} \frac{\partial r_i}{\partial x_k} + r_i \frac{\partial^2 r_i}{\partial x_j \partial x_k} \right) = [J^T J]_{jk} + \sum_{i=1}^{m} r_i [\nabla^2 r_i]_{jk}.$$

(c) Gauss–Newton update: $\Delta x = -(J^T J)^{-1} J^T r(x)$, so

$$x^{(k+1)} = x^{(k)} - (J(x^{(k)})^T J(x^{(k)}))^{-1} J(x^{(k)})^T r(x^{(k)}).$$

The approximation is good when the residuals $r_i(x)$ are small near the solution (i.e., the model fits the data well), because the dropped terms $\sum r_i \nabla^2 r_i$ are proportional to the residuals.
(d) The Newton step with the approximate Hessian $H \approx J^T J$ satisfies $H\,\Delta x = -\nabla f = -J^T r$, giving $J^T J\,\Delta x = -J^T r$. These are the *normal equations* for the linearized least-squares subproblem $\min_{\Delta x} \|J\,\Delta x + r\|^2$.

Exercise 3.3 [Hard]
Consider minimizing the logistic regression loss for a single data point (a, b) with $a \in \mathbb{R}^d$, $b \in \{0, 1\}$:

$$f(\theta) = -b \log \sigma(a^T \theta) - (1 - b) \log(1 - \sigma(a^T \theta)),$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function.

(a) Show that $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

(b) Compute the gradient $\nabla_\theta f(\theta)$ and show it equals $(\sigma(a^T \theta) - b)\, a$.

(c) Compute the Hessian $\nabla_\theta^2 f(\theta)$ and show it equals $\sigma(a^T \theta)(1 - \sigma(a^T \theta))\, a a^T$.

(d) Explain why the Hessian is always positive semidefinite but never positive definite (for $d > 1$). What consequence does this have for Newton's method applied directly?

Solution

(a) $\sigma(z) = (1 + e^{-z})^{-1}$. Then:

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z) \cdot (1 - \sigma(z)).$$

(b) Let $p = \sigma(a^T\theta)$. By chain rule:

$$\nabla_\theta f = -b\,\frac{\sigma'(a^T\theta)}{\sigma(a^T\theta)}\,a - (1-b)\frac{-\sigma'(a^T\theta)}{1-\sigma(a^T\theta)}\,a.$$

Using $\sigma' = p(1-p)$: first term $= -b(1-p)a$, second term $= (1-b)p\,a$. Sum:

$$\nabla_\theta f = [-b + bp + p - bp]\,a = (p-b)\,a = (\sigma(a^T\theta) - b)\,a.$$

(c) $\nabla_\theta f = (\sigma(a^T\theta) - b)\,a$. Differentiating:

$$\nabla_\theta^2 f = \sigma'(a^T\theta)\,aa^T = \sigma(a^T\theta)(1 - \sigma(a^T\theta))\,aa^T.$$

(d) aa^T is PSD (rank-1), and $\sigma(z)(1-\sigma(z)) > 0$ for all finite z, so the Hessian is PSD. However, $\text{rank}(aa^T) = 1$, so for $d > 1$ the Hessian is singular (eigenvalue 0 with multiplicity $d-1$). Newton's method cannot be applied directly because the Hessian is not invertible. This motivates regularized Newton methods (e.g., adding λI) or using the full dataset Hessian $\sum_i \sigma_i(1-\sigma_i)a_i a_i^T$, which can be full rank if enough data points span $\mathbb{R}^d$.

Quasi-Newton Methods

Exercise 4.1 [Easy]
Quasi-Newton methods approximate the inverse Hessian $[\nabla^2 f]^{-1}$ by a matrix H_k that is updated at each step, avoiding the cost of computing and inverting the true Hessian.

(a) The **secant condition** requires the approximation B_{k+1} (approximating $\nabla^2 f$) to satisfy:

$$B_{k+1}s_k = y_k,$$

where $s_k = x^{(k+1)} - x^{(k)}$ and $y_k = \nabla f(x^{(k+1)}) - \nabla f(x^{(k)})$. Explain why this condition is a natural requirement by relating it to the first-order Taylor expansion of ∇f around $x^{(k+1)}$.

(b) For a quadratic $f(x) = \frac{1}{2}x^T Ax - b^T x$, show that the secant condition $B_{k+1}s_k = y_k$ is *exactly* satisfied by $B_{k+1} = A$ (the true Hessian), regardless of s_k.

(c) The quasi-Newton update direction is $d_k = -H_k \nabla f(x^{(k)})$ where $H_k \approx [\nabla^2 f]^{-1}$. If $H_k = I$, what method does this reduce to?

Solution

(a) The first-order expansion of ∇f around $x^{(k+1)}$:

$$\nabla f(x^{(k)}) \approx \nabla f(x^{(k+1)}) + \nabla^2 f(x^{(k+1)})(x^{(k)} - x^{(k+1)}).$$

Rearranging: $\nabla^2 f(x^{(k+1)})s_k \approx y_k$. The secant condition enforces that B_{k+1} mimics this first-order relationship between the change in gradient and the change in position.
(b) For $f(x) = \frac{1}{2}x^T Ax - b^T x$: $\nabla f(x) = Ax - b$. Then $y_k = Ax^{(k+1)} - b - (Ax^{(k)} - b) = As_k$. So $B_{k+1} = A$ satisfies $As_k = y_k$ for any s_k.
(c) With $H_k = I$: $d_k = -\nabla f(x^{(k)})$. This is standard gradient descent.

Exercise 4.2 [Medium]
The **BFGS** update for the inverse Hessian approximation H_k is:

$$H_{k+1} = \left(I - \rho_k s_k y_k^T\right) H_k \left(I - \rho_k y_k s_k^T\right) + \rho_k s_k s_k^T,$$

where $\rho_k = \frac{1}{y_k^T s_k}$.

(a) Verify that H_{k+1} satisfies the inverse secant condition $H_{k+1}y_k = s_k$.

(b) Show that if H_k is symmetric, then H_{k+1} is also symmetric.

(c) In a neural network with $d = 10^6$ parameters, what is the memory cost of storing H_k as a dense matrix? Explain why **L-BFGS** (limited-memory BFGS), which stores only the last m pairs $\{s_k, y_k\}$ instead of the full matrix, is preferred in such settings.

Solution

(a) Compute $H_{k+1}y_k$:

$$(I - \rho_k s_k y_k^T)H_k(I - \rho_k y_k s_k^T)y_k + \rho_k s_k s_k^T y_k.$$

First: $(I - \rho_k y_k s_k^T)y_k = y_k - \rho_k y_k(s_k^T y_k) = y_k - y_k = 0$ (since $\rho_k s_k^T y_k = 1$). So the first term vanishes. Second: $\rho_k s_k(s_k^T y_k) = s_k$. Thus $H_{k+1}y_k = s_k$. ✓
(b) $H_{k+1}^T = (I - \rho_k y_k s_k^T)^T H_k^T (I - \rho_k s_k y_k^T)^T + \rho_k(s_k s_k^T)^T$. Since $H_k^T = H_k$ (symmetric), ρ_k is a scalar, and $(s_k s_k^T)^T = s_k s_k^T$:

$$H_{k+1}^T = (I - \rho_k s_k y_k^T)H_k(I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T = H_{k+1}.$$

(c) A dense $d \times d$ matrix with $d = 10^6$ requires 10^{12} entries ≈ 8 TB in double precision. This is completely infeasible. L-BFGS stores only m pairs of d-vectors (typically $m = 5$–20), requiring $2md \approx 2 \times 10 \times 10^6 = 2 \times 10^7$ entries ≈ 160 MB, which is manageable. The matrix-vector product $H_k \nabla f$ is computed implicitly via a two-loop recursion using these stored pairs.

Exercise 4.3 [Hard]
Consider the **Broyden family** of quasi-Newton updates, which interpolates between the BFGS and DFP updates via parameter $\phi \in [0, 1]$. The BFGS update is at $\phi = 0$ and DFP at $\phi = 1$.

(a) The DFP update for H_k (approximating $[\nabla^2 f]^{-1}$) is:

$$H_{k+1}^{\text{DFP}} = H_k - \frac{H_k y_k y_k^T H_k}{y_k^T H_k y_k} + \frac{s_k s_k^T}{y_k^T s_k}.$$

Verify the inverse secant condition $H_{k+1}^{\text{DFP}} y_k = s_k$.

(b) Compare the computational cost per iteration of: (i) gradient descent, (ii) Newton's method, (iii) a quasi-Newton method (BFGS). Express costs in terms of d (the dimension) and the cost of evaluating ∇f.

(c) In practice, BFGS typically outperforms DFP. One explanation is *self-correcting behavior*: errors in H_k are corrected over subsequent iterations. Suppose the current $H_k = I$ (a poor approximation to $[\nabla^2 f]^{-1}$). After one BFGS update, H_{k+1} incorporates the curvature information from (s_k, y_k). Argue that after d linearly independent steps $s_0, \ldots, s_{d-1}$, BFGS on a quadratic function will recover the exact inverse Hessian A^{-1}.

Solution

(a) $H_{k+1}^{\text{DFP}} y_k = H_k y_k - \frac{H_k y_k (y_k^T H_k y_k)}{y_k^T H_k y_k} + \frac{s_k (s_k^T y_k)}{y_k^T s_k}$. First two terms: $H_k y_k - H_k y_k = 0$. Third term: $\frac{s_k (y_k^T s_k)}{y_k^T s_k} = s_k$. Hence $H_{k+1}^{\text{DFP}} y_k = s_k$. ✓

(b) Per iteration costs:

- **Gradient descent**: $O(d)$ for the update $x \leftarrow x - t\nabla f$ plus the cost of evaluating ∇f.
- **Newton's method**: $O(d^2)$ for computing the Hessian, $O(d^3)$ for solving the linear system $\nabla^2 f \cdot \Delta x = -\nabla f$, plus the cost of ∇f.
- **BFGS**: $O(d^2)$ for the matrix-vector product $H_k \nabla f$ and the rank-two update of H_k, plus the cost of ∇f. No $O(d^3)$ linear solve needed.

(c) For a quadratic $f(x) = \frac{1}{2} x^T A x - b^T x$, we have $y_k = A s_k$ for all k. After d steps with linearly independent $\{s_k\}$, define $S = [s_0 | \cdots | s_{d-1}]$ and $Y = [y_0 | \cdots | y_{d-1}] = AS$. The d secant conditions $H_d y_k = s_k$ for $k = 0, \ldots, d-1$ can be written as $H_d Y = S$, i.e., $H_d A S = S$. Since S is invertible (d linearly independent columns), we get $H_d A = I$, hence $H_d = A^{-1}$. BFGS recovers the exact inverse Hessian in at most d steps on a quadratic, a property known as the *finite termination* or *hereditary* property of quasi-Newton methods.

Stochastic Gradient Descent

Exercise 5.1 [Easy]
Consider the finite-sum objective $F(x) = \frac{1}{n} \sum_{i=1}^n f_i(x)$ and the SGD update $x^{(k+1)} = x^{(k)} - \alpha_k \nabla f_{i_k}(x^{(k)})$, where i_k is drawn uniformly at random from $\{1, \ldots, n\}$.

(a) Show that the stochastic gradient is an unbiased estimator of the true gradient:

$$\mathbb{E}_{i_k}[\nabla f_{i_k}(x^{(k)}) \mid x^{(k)}] = \nabla F(x^{(k)}).$$

(b) For the mini-batch variant with batch $B \subset \{1, \ldots, n\}$ of size $|B|$, the stochastic gradient is $g_B = \frac{1}{|B|} \sum_{j \in B} \nabla f_j(x)$. Show that this is also an unbiased estimator.

(c) If $\text{Var}(\nabla f_i(x)) = \sigma^2$ for each i, what is the variance of the mini-batch estimator g_B? How does increasing the batch size affect variance?

Solution

(a) Since i_k is drawn uniformly:

$$\mathbb{E}_{i_k}[\nabla f_{i_k}(x)] = \sum_{r=1}^{n} \nabla f_r(x) \cdot P(i_k = r) = \sum_{r=1}^{n} \nabla f_r(x) \cdot \frac{1}{n} = \frac{1}{n} \sum_{r=1}^{n} \nabla f_r(x) = \nabla F(x).$$

(b) By linearity of expectation and the result from (a):

$$\mathbb{E}[g_B] = \frac{1}{|B|} \sum_{j \in B} \mathbb{E}[\nabla f_j(x)] = \frac{1}{|B|} \cdot |B| \cdot \nabla F(x) = \nabla F(x).$$

(c) Since the samples are drawn independently:

$$\mathrm{Var}(g_B) = \frac{1}{|B|^2} \sum_{j \in B} \mathrm{Var}(\nabla f_j(x)) = \frac{|B| \cdot \sigma^2}{|B|^2} = \frac{\sigma^2}{|B|}.$$

Increasing the batch size $|B|$ reduces the variance by a factor of $1/|B|$. However, the computational cost per iteration scales linearly with $|B|$, so there is a trade-off between per-step noise reduction and computational cost.

Exercise 5.2 [Medium]
Consider $F(x) = \frac{1}{n} \sum_{i=1}^{n} f_i(x)$, where F has L-Lipschitz continuous gradient and we assume the bounded variance condition $\mathbb{E}[\|\nabla f_i(x)\|^2] \leq \sigma^2$.

(a) Starting from the descent lemma

$$F(x^{(k+1)}) \leq F(x^{(k)}) - \alpha_k \langle \nabla F(x^{(k)}), g_k \rangle + \frac{L\alpha_k^2}{2} \|g_k\|^2,$$

where $g_k = \nabla f_{i_k}(x^{(k)})$, take the conditional expectation $\mathbb{E}[\cdot \mid x^{(k)}]$ on both sides. Show that

$$\mathbb{E}[F(x^{(k+1)}) \mid x^{(k)}] \leq F(x^{(k)}) - \alpha_k \|\nabla F(x^{(k)})\|^2 + \frac{L\alpha_k^2}{2} \sigma^2.$$

(b) Interpret each term on the right-hand side: what does $-\alpha_k \|\nabla F(x^{(k)})\|^2$ represent and what does $\frac{L\alpha_k^2}{2}\sigma^2$ represent?

(c) Using a constant step size α, suppose we run SGD until convergence. Show that the iterates satisfy

$$\min_{k=1,\ldots,T} \mathbb{E}[\|\nabla F(x^{(k)})\|^2] \leq \frac{F(x^{(0)}) - F^*}{\alpha T} + \frac{L\alpha\sigma^2}{2}.$$

Explain why SGD with constant step size cannot converge exactly to a critical point but only to a neighborhood.

Solution

(a) Taking $\mathbb{E}[\cdot|x^{(k)}]$:

- $\mathbb{E}[\langle \nabla F(x^{(k)}), g_k\rangle|x^{(k)}] = \langle \nabla F(x^{(k)}), \mathbb{E}[g_k|x^{(k)}]\rangle = \|\nabla F(x^{(k)})\|^2$ (unbiasedness).
- $\mathbb{E}[\|g_k\|^2|x^{(k)}] \le \sigma^2$ (bounded second moment assumption).

Substituting: $\mathbb{E}[F(x^{(k+1)})|x^{(k)}] \le F(x^{(k)}) - \alpha_k\|\nabla F(x^{(k)})\|^2 + \frac{L\alpha_k^2}{2}\sigma^2$.
(b) The term $-\alpha_k\|\nabla F(x^{(k)})\|^2$ is the *progress term*: it captures the expected decrease from moving along the true gradient direction. The term $\frac{L\alpha_k^2}{2}\sigma^2$ is the *noise penalty*: it captures the expected increase due to using a noisy gradient estimate. Progress dominates when the gradient is large; noise dominates near critical points.
(c) Rearranging and summing from $k = 0$ to $T - 1$:

$$\sum_{k=0}^{T-1} \alpha\|\nabla F(x^{(k)})\|^2 \le F(x^{(0)}) - \mathbb{E}[F(x^{(T)})] + \frac{L\alpha^2\sigma^2 T}{2} \le F(x^{(0)}) - F^* + \frac{L\alpha^2\sigma^2 T}{2}.$$

Dividing by αT:

$$\frac{1}{T}\sum_{k=0}^{T-1} \mathbb{E}[\|\nabla F(x^{(k)})\|^2] \le \frac{F(x^{(0)}) - F^*}{\alpha T} + \frac{L\alpha\sigma^2}{2}.$$

The minimum is bounded by the average. As $T \to \infty$, the first term vanishes but the second term $\frac{L\alpha\sigma^2}{2}$ remains. SGD with constant step size converges only to a neighborhood of size $O(\alpha\sigma^2)$ around a critical point. To converge exactly, one needs a decreasing step size schedule.

Exercise 5.3 [Hard]
Let $F(x) = \frac{1}{n}\sum_{i=1}^n f_i(x)$ be μ-strongly convex with L-Lipschitz continuous gradient, and suppose we run SGD with constant step size $0 < \alpha \le \frac{1}{2L}$.

(a) Using the descent lemma from Exercise 5.2(a) and strong convexity, show that

$$\mathbb{E}[F(x^{(k+1)})] - F^* \le (1 - 2\alpha\mu)\left(\mathbb{E}[F(x^{(k)})] - F^*\right) + \frac{L\alpha^2\sigma^2}{2}.$$

(Hint: use the PL condition from Exercise 2.2 to replace $\|\nabla F\|^2$ with $F - F^*$.)

(b) By iterating the recurrence in (a), show that

$$\mathbb{E}[F(x^{(k)})] - F^* \le (1 - 2\alpha\mu)^k(F(x^{(0)}) - F^*) + \frac{L\alpha\sigma^2}{4\mu}.$$

(Hint: use $\sum_{s=0}^\infty r^s = \frac{1}{1-r}$ for $|r| < 1$.)

(c) Interpret the two terms on the right-hand side. A common training strategy is to run SGD with step size α until $\mathbb{E}[F(x^{(k)})] - F^* \approx \frac{L\alpha\sigma^2}{2\mu}$, then halve α and repeat. Explain why this *step size schedule* is effective and relate it to the common practice of learning rate decay in deep learning.

Solution

(a) From Exercise 5.2(a): $\mathbb{E}[F(x^{(k+1)})|x^{(k)}] \leq F(x^{(k)}) - \alpha\|\nabla F(x^{(k)})\|^2 + \frac{L\alpha^2\sigma^2}{2}$. Since F is μ-strongly convex, the PL condition gives $\|\nabla F(x^{(k)})\|^2 \geq 2\mu(F(x^{(k)}) - F^*)$. Substituting:

$$\mathbb{E}[F(x^{(k+1)})|x^{(k)}] \leq F(x^{(k)}) - 2\alpha\mu(F(x^{(k)}) - F^*) + \frac{L\alpha^2\sigma^2}{2}.$$

Subtracting F^* and taking full expectation:

$$\mathbb{E}[F(x^{(k+1)})] - F^* \leq (1 - 2\alpha\mu)(\mathbb{E}[F(x^{(k)})] - F^*) + \frac{L\alpha^2\sigma^2}{2}.$$

(b) Let $\delta_k = \mathbb{E}[F(x^{(k)})] - F^*$ and $r = 1 - 2\alpha\mu \in (0, 1)$ and $c = \frac{L\alpha^2\sigma^2}{2}$. Then $\delta_{k+1} \leq r\delta_k + c$. Iterating:

$$\delta_k \leq r^k\delta_0 + c\sum_{s=0}^{k-1} r^s \leq r^k\delta_0 + \frac{c}{1-r} = (1-2\alpha\mu)^k(F(x^{(0)}) - F^*) + \frac{L\alpha^2\sigma^2}{2 \cdot 2\alpha\mu} = (1-2\alpha\mu)^k\delta_0 + \frac{L\alpha\sigma^2}{4\mu}.$$

(c) The first term $(1 - 2\alpha\mu)^k\delta_0$ decays exponentially (linear convergence) and represents the optimization of the deterministic problem. The second term $\frac{L\alpha\sigma^2}{4\mu}$ is the *noise floor*: the steady-state error due to gradient noise, proportional to α.
Once the first term is negligible (k large), the suboptimality is dominated by the noise floor. Halving α halves the noise floor, allowing further progress. Each halving triggers a new phase of linear convergence toward the reduced noise floor. This is exactly learning rate decay in practice: large initial learning rate for fast progress, then periodic reductions to fine-tune near the optimum.

Mixed Challenging Exercises

Exercise 6.1 [Hard]
Consider training a model by minimizing $F(\theta) = \frac{1}{n}\sum_{i=1}^n \ell(h_\theta(x_i), y_i)$, where ℓ is a smooth loss function.

(a) Suppose ∇F is L-Lipschitz and F satisfies the PL condition with constant μ. Gradient descent with step size $t = 1/L$ gives:

$$F(\theta^{(k)}) - F^* \leq \left(1 - \frac{\mu}{L}\right)^k (F(\theta^{(0)}) - F^*).$$

SGD with step size $\alpha \leq \frac{1}{2L}$ gives:

$$\mathbb{E}[F(\theta^{(k)})] - F^* \leq (1 - 2\alpha\mu)^k (F(\theta^{(0)}) - F^*) + \frac{L\alpha\sigma^2}{4\mu}.$$

Suppose the cost of computing ∇F (full gradient) is n times the cost of computing one ∇f_i (stochastic gradient). To reach accuracy ε, GD needs $O(\frac{L}{\mu}\log\frac{1}{\varepsilon})$ iterations, each costing n gradient evaluations. For SGD with constant step size $\alpha = \frac{\varepsilon\mu}{L\sigma^2}$, how many stochastic gradient evaluations are needed? For what values of n is SGD cheaper?

(b) The **Gauss–Newton** method for $F(\theta) = \frac{1}{2n}\|r(\theta)\|^2$ (where r is the vector of residuals) uses $H_k \approx \frac{1}{n}J_k^T J_k$ instead of the true Hessian. In what sense is this a quasi-Newton method? Does it satisfy the secant condition in general?

(c) A practitioner observes that SGD training loss decreases quickly at first but then plateaus, oscillating around a value. Gradient descent from the same initialization converges to a lower loss but takes much longer. Using the theory from this exercise set, explain both observations quantitatively.

Solution

(a) GD cost: $O\left(n \cdot \frac{L}{\mu} \log \frac{1}{\varepsilon}\right)$ stochastic gradient evaluations.
For SGD: with $\alpha = \frac{\varepsilon\mu}{L\sigma^2}$, the noise floor is $\frac{L\alpha\sigma^2}{4\mu} = \frac{\varepsilon}{4} < \varepsilon$. The exponential term requires $(1-2\alpha\mu)^k \leq \varepsilon/(F_0 - F^*)$, so $k = O(\frac{1}{\alpha\mu}\log\frac{1}{\varepsilon}) = O(\frac{L\sigma^2}{\varepsilon\mu^2}\log\frac{1}{\varepsilon})$. Each iteration costs 1 stochastic gradient, so total cost: $O\left(\frac{L\sigma^2}{\varepsilon\mu^2}\log\frac{1}{\varepsilon}\right)$.
SGD is cheaper when $\frac{L\sigma^2}{\varepsilon\mu^2} < n \cdot \frac{L}{\mu}$, i.e., when $n > \frac{\sigma^2}{\varepsilon\mu}$. For large datasets ($n$ large) and moderate accuracy ε, SGD is much cheaper.
(b) Gauss–Newton uses a fixed approximation of the Hessian (the first-order part J^TJ) rather than an iteratively updated one. Unlike classical quasi-Newton (BFGS, DFP), it does not enforce the secant condition. It is "quasi-Newton" in the loose sense that it replaces the true Hessian with an approximation, but it differs from the BFGS family in that the approximation comes from the problem structure (dropping second-order residual terms) rather than from accumulated gradient difference information.
(c) SGD's fast initial decrease comes from the linear convergence rate $(1-2\alpha\mu)^k$, which is fast when $F - F^*$ is large. The plateau with oscillations is the noise floor $\frac{L\alpha\sigma^2}{4\mu}$: constant step size prevents convergence below this level, and the stochastic noise causes oscillation around it. GD has no noise (it uses the exact gradient), so it converges all the way to F^* with rate $(1-\mu/L)^k$, but each iteration is n times more expensive. The total wall-clock time is longer because $n \cdot \frac{L}{\mu}\log\frac{1}{\varepsilon}$ can be much larger than the SGD cost for moderate ε.

Exercise 6.2 [Hard]
Let $f : \mathbb{R}^d \to \mathbb{R}$ be twice continuously differentiable with $\nabla^2 f(x) \succeq mI$ for all x (strongly convex) and $\|\nabla^2 f(x) - \nabla^2 f(y)\| \leq M\|x-y\|$ (Lipschitz continuous Hessian).

(a) Newton's method has *quadratic convergence* near the optimum: there exists $\delta > 0$ such that if $\|x^{(k)} - x^*\| < \delta$, then

$$\|x^{(k+1)} - x^*\| \leq \frac{M}{2m}\|x^{(k)} - x^*\|^2.$$

Using the Taylor expansion of ∇f around x^*, derive this bound. (Hint: $x^{(k+1)} - x^* = x^{(k)} - x^* - [\nabla^2 f(x^{(k)})]^{-1}\nabla f(x^{(k)})$. Use $\nabla f(x^*) = 0$.)

(b) Quadratic convergence means the number of correct digits roughly doubles each iteration. If $\|x^{(0)} - x^*\| = 0.1$ and $M/(2m) = 1$, how many Newton steps are needed for $\|x^{(k)} - x^*\| < 10^{-16}$?

(c) Compare this to gradient descent, which achieves linear convergence $\|x^{(k)} - x^*\| \leq (1-m/L)^k\|x^{(0)} - x^*\|$. For $m/L = 0.01$, how many GD steps are needed for the same accuracy? Comment on the trade-off between per-iteration cost and number of iterations.

Solution

(a) Since $\nabla f(x^*) = 0$:

$$x^{(k+1)} - x^* = x^{(k)} - x^* - [\nabla^2 f(x^{(k)})]^{-1}(\nabla f(x^{(k)}) - \nabla f(x^*)).$$

Using the integral form: $\nabla f(x^{(k)}) - \nabla f(x^*) = \int_0^1 \nabla^2 f(x^* + t(x^{(k)} - x^*))\, dt \cdot (x^{(k)} - x^*)$. Thus:

$$x^{(k+1)} - x^* = [\nabla^2 f(x^{(k)})]^{-1} \left(\nabla^2 f(x^{(k)}) - \int_0^1 \nabla^2 f(x^* + t(x^{(k)} - x^*))\, dt \right) (x^{(k)} - x^*).$$

Taking norms: $\|[\nabla^2 f(x^{(k)})]^{-1}\| \leq 1/m$, and

$$\left\| \nabla^2 f(x^{(k)}) - \nabla^2 f(x^* + t(x^{(k)} - x^*)) \right\| \leq M(1-t)\|x^{(k)} - x^*\|.$$

Integrating: $\int_0^1 M(1-t)\, dt = M/2$. Therefore $\|x^{(k+1)} - x^*\| \leq \frac{M}{2m}\|x^{(k)} - x^*\|^2$.
(b) $\|x^{(k)} - x^*\| \leq (0.1)^{2^k}$. We need $(0.1)^{2^k} < 10^{-16}$, i.e., $2^k > 16$, so $k \geq 5$. Only 5 Newton steps for 16 digits of accuracy.
(c) Need $(1-0.01)^k \cdot 0.1 < 10^{-16}$, so $(0.99)^k < 10^{-15}$, giving $k > \frac{15 \ln 10}{\ln(1/0.99)} \approx \frac{34.5}{0.01005} \approx 3{,}433$ steps.
Newton: 5 iterations but each costs $O(d^3)$. GD: $\sim 3{,}400$ iterations but each costs $O(d)$. Newton wins when d is moderate and high accuracy is needed. GD (or quasi-Newton) wins when d is very large and moderate accuracy suffices, which is the typical regime in machine learning.